

Policy Optimization: Actor-Critic to PPO

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 18, 2026

1. Recap
2. Learning Outcomes
3. Actor-Critic Methods
4. A2C and A3C
5. Trust Region Motivation
6. Surrogate Objective
7. Theoretical Foundations
8. Trust Region Policy Optimization (TRPO)
9. Proximal Policy Optimization (PPO)
10. Group Relative Policy Optimization (GRPO)
11. Summary
12. References

Policy Optimization: **Recap (Day 3)**

- ▶ The policy gradient theorem gives us:

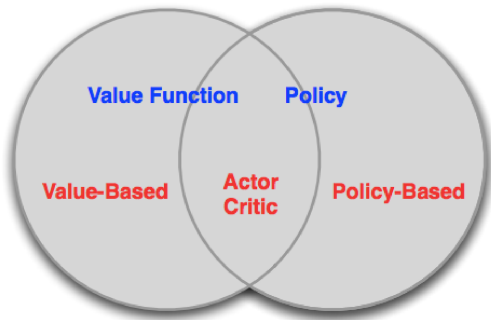
$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t) \right]$$

- ▶ $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ is the **advantage function** — the optimal baseline
- ▶ Using A^{π} instead of raw returns reduces variance without introducing bias
- ▶ **Unresolved question:** how do we estimate A^{π} in practice?
- ▶ Day 3 noted: we only need V^{π} , since the TD residual $\delta_t = r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$ approximates A^{π}

- ▶ **Today we cash that promise.**
- ▶ We learn a critic V_ϕ alongside the policy — this is the **Actor-Critic** architecture
- ▶ The critic is trained via TD learning; the TD residual δ_t serves as an online, low-variance estimate of A^π
- ▶ Then we scale up: A2C/A3C $\rightarrow$ trust region (TRPO) $\rightarrow$ clipped objective (PPO)

- ▶ Explain why a learned critic reduces variance and enables online updates (Actor-Critic)
- ▶ Derive the TD-residual advantage estimator and connect it to the Bellman equation
- ▶ Describe A2C and A3C at a high level and identify when each is preferred
- ▶ State the TRPO trust-region constraint and why it prevents policy collapse
- ▶ Implement the PPO clipped objective and explain how clipping enforces the trust region

Policy Optimization: **Actor-Critic** **Methods**



- ▶ **Value-based** methods (Days 1–2): learn V or Q , derive policy implicitly
- ▶ **Policy-based** methods (Day 3): learn π_θ directly, no value function
- ▶ **Actor-Critic** sits at the intersection: learn *both* a policy *and* a value function

- ▶ Day 3: subtracting a **state-dependent baseline** $b(s)$ reduces variance with zero bias:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_t [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q^{\pi}(s_t, a_t) - b(s_t))]$$

- ▶ The best choice of baseline is $b(s) = V^{\pi}(s)$, giving $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$

- ▶ Day 3: subtracting a **state-dependent baseline** $b(s)$ reduces variance with zero bias:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_t [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q^{\pi}(s_t, a_t) - b(s_t))]$$

- ▶ The best choice of baseline is $b(s) = V^{\pi}(s)$, giving $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$
- ▶ But we don't know V^{π} — we have to **learn it**
- ▶ This turns the baseline into a **critic**: a second network $V_{\phi}(s)$ trained alongside the actor π_{θ}
- ▶ **Actor** $\pi_{\theta}(a|s)$: selects actions, updated by policy gradient
- ▶ **Critic** $V_{\phi}(s)$: evaluates states, updated by TD learning

- ▶ Given a critic V_ϕ , define the **TD residual**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- ▶ When $V_\phi = V^\pi$ (a perfect critic), the **expected** TD residual equals the advantage:

$$\begin{aligned}\mathbb{E}[\delta_t \mid s_t, a_t] &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}}[V^\pi(s_{t+1})] - V^\pi(s_t) \\ &= Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)\end{aligned}$$

where $r(s_t, a_t) = \mathbb{E}[r_t \mid s_t, a_t]$ is the expected reward function. In practice we use the observed r_t and a learned V_ϕ , so δ_t is an approximation of A^π .

- ▶ Given a critic V_ϕ , define the **TD residual**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- ▶ When $V_\phi = V^\pi$ (a perfect critic), the **expected** TD residual equals the advantage:

$$\begin{aligned}\mathbb{E}[\delta_t \mid s_t, a_t] &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}}[V^\pi(s_{t+1})] - V^\pi(s_t) \\ &= Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)\end{aligned}$$

where $r(s_t, a_t) = \mathbb{E}[r_t \mid s_t, a_t]$ is the expected reward function. In practice we use the observed r_t and a learned V_ϕ , so δ_t is an approximation of A^π .

- ▶ So we can use δ_t as a **low-variance, online** estimate of A^π — no full trajectory needed
- ▶ This is what Day 3 deferred: we now have a way to estimate A^π at every timestep
- ▶ **Critic update:** minimize the TD error squared: $\mathcal{L}(\phi) = \mathbb{E}_t[\delta_t^2]$

- ▶ Between 1-step TD and full Monte Carlo lies a whole family of **n -step advantage estimators**:

$$\hat{A}_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_\phi(s_{t+n})}_{n\text{-step return}} - V_\phi(s_t)$$

- Small n : low variance, high bias; large n : high variance, low bias

- ▶ Between 1-step TD and full Monte Carlo lies a whole family of **n -step advantage estimators**:

$$\hat{A}_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_\phi(s_{t+n})}_{n\text{-step return}} - V_\phi(s_t)$$

- Small n : low variance, high bias; large n : high variance, low bias
- ▶ Each $\hat{A}_t^{(n)}$ telescopes into a sum of TD residuals: $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$
- ▶ **GAE** avoids picking a single n : take an **exponentially-weighted average** over all n with decay λ :

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

- ▶ λ smoothly tunes the bias–variance trade-off (next slide: the limits)

- ▶ GAE gives a single knob λ that interpolates the bias–variance trade-off:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

- ▶ $\lambda = 0$: pure 1-step TD — low variance, biased
- ▶ $\lambda = 1$: full Monte Carlo ($G_t - V_\phi(s_t)$) — unbiased, high variance
- ▶ $\lambda \approx 0.95$: standard in PPO implementations

- ▶ GAE gives a single knob λ that interpolates the bias–variance trade-off:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

- ▶ $\lambda = 0$: pure 1-step TD — low variance, biased
- ▶ $\lambda = 1$: full Monte Carlo ($G_t - V_\phi(s_t)$) — unbiased, high variance
- ▶ $\lambda \approx 0.95$: standard in PPO implementations
- ▶ We write $\hat{A}_t$ for short: a **practical estimate** of the true advantage $A^\pi(s_t, a_t)$ from Day 1
- ▶ **Used by PPO in the lab** — we revisit this when implementing the algorithm


```
Initialise actor  $\theta$ , critic  $\phi$ ;  
for each episode do  
  observe initial state  $s_0$ ;  
  for  $t = 0, 1, 2, \dots$  do  
    sample  $a_t \sim \pi_\theta(\cdot | s_t)$ ;  
    take  $a_t$ , observe  $r_t, s_{t+1}$ ;  
     $\delta_t \leftarrow r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ ;  
     $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t$ ;  
     $\phi \leftarrow \phi - \alpha_\phi \delta_t \nabla_\phi V_\phi(s_t)$ ;  
  end  
end
```

Algorithm: One-step Actor-Critic (on-line)

- ▶ **Actor** steps along the policy gradient, weighted by δ_t
- ▶ **Critic** regresses $V_\phi(s_t)$ toward $r_t + \gamma V_\phi(s_{t+1})$
- ▶ The same δ_t drives *both* updates — no full trajectory needed
- ▶ This is the pure **TD** form; batched A2C/PPO swap the single δ_t for a **GAE** estimate $\hat{A}_t$ over minibatches

- ▶ **Variance reduction vs REINFORCE:** the TD residual δ_t is a much lower-variance signal than $G_t - b$ because it bootstraps off V_ϕ rather than waiting for the full trajectory
- ▶ **Online updates:** no need to wait until the end of an episode — the critic provides a training signal at every timestep
- ▶ **Bootstrapping:** propagating value estimates forward allows the agent to learn from partial trajectories and even in continuing (non-episodic) tasks

- ▶ **Variance reduction vs REINFORCE:** the TD residual δ_t is a much lower-variance signal than $G_t - b$ because it bootstraps off V_ϕ rather than waiting for the full trajectory
- ▶ **Online updates:** no need to wait until the end of an episode — the critic provides a training signal at every timestep
- ▶ **Bootstrapping:** propagating value estimates forward allows the agent to learn from partial trajectories and even in continuing (non-episodic) tasks
- ▶ **Trade-off:** if V_ϕ is a bad critic, the TD residual is a biased advantage estimate — the actor is being guided by a noisy critic
- ▶ Both networks must be trained together carefully; instability can arise if the critic lags the actor
- ▶ This is the bias-variance trade-off formalised by GAE

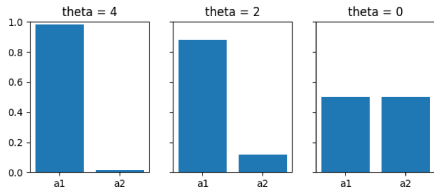
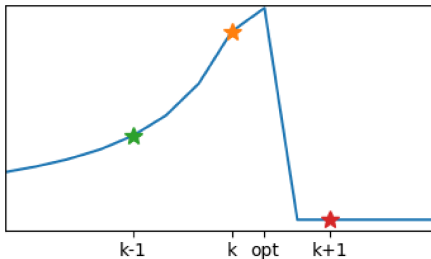
- ▶ **Key idea:** run N parallel actors sharing the same θ and ϕ
- ▶ All actors step simultaneously; gradients are aggregated and a single synchronous update is applied
- ▶ The shared critic sees diverse experience from all environments — more stable training signal
- ▶ **Policy loss:** $\mathcal{L}^{\pi}(\theta) = -\mathbb{E}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$
- ▶ **Value loss:** $\mathcal{L}^V(\phi) = \mathbb{E}_t [\delta_t^2]$
- ▶ A2C is what most modern Actor-Critic implementations actually use
- ▶ Parallel actors de-correlate experience without a replay buffer (on-policy)

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ
- ▶ **Going forward:** we use A2C as the basis for PPO; A3C is historical context

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ
- ▶ **Going forward:** we use A2C as the basis for PPO; A3C is historical context
- ▶ But parallel actors only *accelerate* data collection — they do not fix the underlying issue that **a single gradient step can still destroy the policy**. That motivates the next section.



- ▶ A large gradient step in *parameter space* can cause a catastrophically large shift in *policy space*
- ▶ Once a policy collapses (e.g., always picks one bad action), the agent collects bad data and the policy is hard to recover
- ▶ We need to control how much the policy distribution changes, not just how much θ changes

- ▶ **Relative policy performance identity:** the improvement of π_θ over $\pi_{\theta_{\text{old}}}$ can be written as:

$$\mathcal{J}(\theta) - \mathcal{J}(\theta_{\text{old}}) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t \geq 0} \gamma^t A^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right]$$

- ▶ Since we collect data under $\pi_{\theta_{\text{old}}}$, re-weight via importance sampling. Define the probability ratio:¹

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- ▶ This gives the **surrogate objective**:

$$\mathcal{L}(\theta) = \mathbb{E}_t[r_t(\theta) A^{\pi_{\theta_{\text{old}}}}(s_t, a_t)]$$

¹ $r_t(\theta)$ here is the probability ratio, not the reward r_t from earlier lectures — distinguished by the θ argument.

- ▶ **Problem:** if π_θ drifts far from $\pi_{\theta_{\text{old}}}$, the ratio $r_t(\theta)$ explodes and the estimate becomes unreliable
- ▶ **Solution: constrain how far π_θ can move** from $\pi_{\theta_{\text{old}}}$
- ▶ Next we make this precise: *why* the surrogate is principled in the first place, and *why* importance sampling specifically demands a constraint — before turning to TRPO and PPO, which both implement the constraint in different ways

- ▶ The identity on the previous slide has a name: the **Performance Difference Lemma**¹

$$\mathcal{J}(\theta) - \mathcal{J}(\theta_{\text{old}}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t A^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right]$$

- ▶ **Why this matters:** it makes the surrogate $\mathcal{L}(\theta)$ a **principled approximation** to the true objective $\mathcal{J}(\theta)$ — not a heuristic
- ▶ When $\pi_{\theta} \approx \pi_{\theta_{\text{old}}}$, the state distributions induced by the two policies are similar, so we can substitute samples from $\pi_{\theta_{\text{old}}}$ for samples from π_{θ} with controlled error
- ▶ As π_{θ} drifts away from $\pi_{\theta_{\text{old}}}$, the approximation degrades. This is the theoretical reason we need a trust region — not just empirical observation that “big steps break things”

¹Lemma originally due to Kakade & Langford (2002); foundation of TRPO in Schulman et al. (2015).

- ▶ Using data from $\pi_{\theta_{\text{old}}}$ requires importance sampling. The IS-weighted estimator has variance

$$\text{Var}\left[\frac{P(x)}{Q(x)} f(x)\right] = \mathbb{E}_{x \sim P}\left[\frac{P(x)}{Q(x)} f(x)^2\right] - \mathbb{E}[f]^2$$

- ▶ The variance **explodes** when the ratio $\frac{P}{Q}$ is large where f^2 is large — a single bad sample dominates the estimate
- ▶ **Concretely:** if $\pi_{\theta_{\text{old}}}(a|s)$ is tiny where $\pi_{\theta}(a|s)$ is large, the ratio $r_t(\theta)$ blows up. One rare sample under the old policy gets enormous weight and swings the entire gradient estimate

²Full derivation in OpenAI's **Spinning Up: TRPO** and Schulman et al. (2015). 

- ▶ Using data from $\pi_{\theta_{\text{old}}}$ requires importance sampling. The IS-weighted estimator has variance

$$\text{Var} \left[\frac{P(x)}{Q(x)} f(x) \right] = \mathbb{E}_{x \sim P} \left[\frac{P(x)}{Q(x)} f(x)^2 \right] - \mathbb{E}[f]^2$$

- ▶ The variance **explodes** when the ratio $\frac{P}{Q}$ is large where f^2 is large — a single bad sample dominates the estimate
- ▶ **Concretely:** if $\pi_{\theta_{\text{old}}}(a|s)$ is tiny where $\pi_{\theta}(a|s)$ is large, the ratio $r_t(\theta)$ blows up. One rare sample under the old policy gets enormous weight and swings the entire gradient estimate
- ▶ For **trajectory-level** IS: $\frac{p(\tau|\pi_{\theta})}{p(\tau|\pi_{\theta_{\text{old}}})} = \prod_{t=0}^T r_t(\theta)$. Small per-step differences **compound multiplicatively** over T steps — the variance is hopeless
- ▶ TRPO and PPO use **single-step ratios** and constrain them to stay near 1 — introduced next²

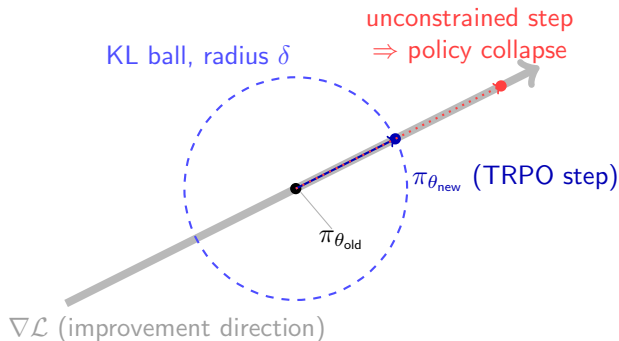
²Full derivation in OpenAI's **Spinning Up: TRPO** and Schulman et al. (2015). 

Policy Optimization: **TRPO**

- ▶ TRPO formalises the constraint: maximize the surrogate objective subject to a KL-divergence bound:

$$\max_{\theta} \mathcal{L}(\theta) \quad \text{s.t.} \quad \mathbb{E}_t[D_{KL}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta$$

- ▶ Keeps the new policy inside a **trust region** in policy space, not parameter space
- ▶ Solved with conjugate gradient + backtracking line search (second-order): largest natural-gradient step that still satisfies the KL constraint
- ▶ **Cost:** needs the Fisher information matrix — expensive and hard to scale. *This is exactly what motivates PPO.*



- Each update stays within a **KL ball** around $\pi_{\theta_{old}}$ (*policy space*, not parameter space): the step climbs $\mathcal{L}$ but is clipped at the boundary, preventing the overshoot that collapses the policy

Policy Optimization: **PPO**

- ▶ TRPO achieves monotonic improvement but requires second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get TRPO-level stability with plain SGD?

- ▶ TRPO achieves monotonic improvement but requires second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get TRPO-level stability with plain SGD?
- ▶ **PPO** (Schulman et al., 2017) proposes two variants:
 - **PPO-KL:** add a KL penalty to the objective and adapt the penalty coefficient β over training
 - **PPO-Clip:** clip the probability ratio $r_t(\theta)$ to prevent large policy updates
- ▶ PPO-Clip is the standard variant — simpler, no hyperparameter tuning on β , works at least as well empirically

► Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

► The **PPO-Clip objective** is:

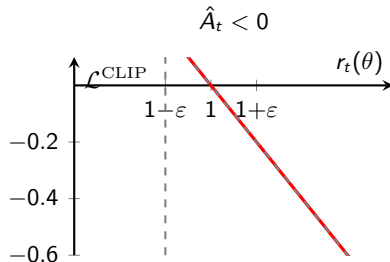
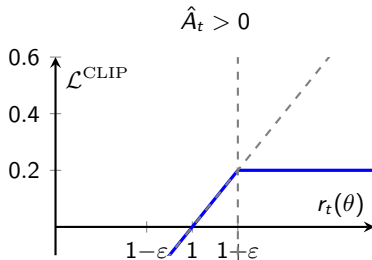
$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

► ε is a small hyperparameter (typically $\varepsilon = 0.2$)

- ▶ Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
- ▶ The **PPO-Clip objective** is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

- ▶ ε is a small hyperparameter (typically $\varepsilon = 0.2$)
- ▶ **Why clip + min?**
 - When $\hat{A}_t > 0$: clipping caps r_t at $1 + \varepsilon$, removing the incentive to increase the ratio further
 - When $\hat{A}_t < 0$: clipping floors r_t at $1 - \varepsilon$, removing the incentive to decrease the ratio further
 - The **min** ensures the objective is a lower bound — we never benefit from going outside $[1 - \varepsilon, 1 + \varepsilon]$
- ▶ This enforces a soft trust region without any second-order computation



- ▶ **Left ($\hat{A}_t > 0$):** benefit grows linearly with r_t until $1 + \epsilon$, then is clipped flat — no reward for pushing the policy further
- ▶ **Right ($\hat{A}_t < 0$):** penalty grows as r_t decreases until $1 - \epsilon$, then is clipped flat
- ▶ Gray dashed line = unclipped surrogate; colored solid = clipped (what PPO optimises)

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

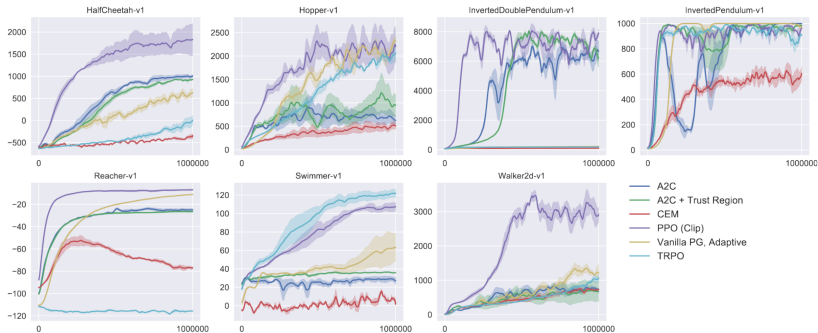
end for

- ▶ **Sample efficiency:** PPO runs multiple SGD epochs over the same rollout — unlike A2C which discards data after one update
- ▶ **Combined loss:** $\mathcal{L}(\theta, \phi) = \mathcal{L}^{CLIP}(\theta) - c_1 \mathcal{L}^V(\phi) + c_2 \mathcal{H}(\pi_\theta)$ (entropy bonus encourages exploration)
- ▶ GAE is used to compute $\hat{A}_t$; the entropy term will reappear in SAC (Day 6)

- ▶ The clipped objective is only half the story — these implementation details matter *as much as* ε , and explain why two “correct” PPO implementations can differ by an order of magnitude²
- ▶ **Advantage normalisation:** standardise $\hat{A}_t$ per minibatch (stabilises gradient scale)
- ▶ **Value-loss clipping:** clip the critic update like the policy
- ▶ **Gradient clipping:** bound global grad-norm (typically 0.5)
- ▶ **KL early stopping:** end the epoch loop once mean KL exceeds a target — a cheap trust-region safeguard
- ▶ **Obs / reward normalisation:** running normalisation keeps inputs and returns well-scaled
- ▶ **Orthogonal init:** small policy-head gain avoids an over-confident initial policy

²Andrychowicz et al. (2020), *What Matters in On-Policy RL?*

PPO Empirical Performance



- ▶ PPO matches or exceeds TRPO with far less complexity; outperforms vanilla PG and CEM on most benchmarks
- ▶ This is the algorithm you will implement in the lab³

³Schulman, Wolski, Dhariwal, Radford, Klimov, 2017

Policy Optimization: **GRPO**

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely
- ▶ Per prompt/state, sample a **group** of G outputs under $\pi_{\theta_{\text{old}}}$, scored with rewards R_i
- ▶ Use the **group mean as the baseline** — a Monte Carlo advantage, no value function:

$$\hat{A}_i = \frac{R_i - \overbrace{\text{mean}(R_1, \dots, R_G)}^{\text{the "Group Relative" baseline}}}{\underbrace{\text{std}(R_1, \dots, R_G)}_{\text{per-group advantage normalisation}}}$$

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is exactly the per-minibatch advantage normalisation from the PPO-in-Practice slide, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is exactly the per-minibatch advantage normalisation from the PPO-in-Practice slide, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser
- ▶ Plug $\hat{A}_i$ into the *same* PPO-Clip objective (plus a KL term to a reference policy)
- ▶ **Why it matters:** no critic $\Rightarrow$ no two-network instability and $\approx$ half the memory
- ▶ Since 2024 GRPO is the default for **LLM RL** (DeepSeek-R1, many open recipes) — revisited in **Day 9 (RLHF)**, where the reward is a learned preference model

Policy Optimization: **Summary**

REINFORCE $\rightarrow$ +baseline $\rightarrow$ +critic (A2C) $\rightarrow$ +GAE $\rightarrow$ +trust region (TRPO) $\rightarrow$ +clipping (PPO) $\rightarrow$ -critic (GRPO)

Algorithm	On-policy?	Variance	Sample eff.	Complexity
REINFORCE	Yes	High	Low	Simple
A2C	Yes	Medium	Medium	Moderate
TRPO	Yes	Medium	Medium	High
PPO	Yes	Medium	Medium	Low
GRPO	Yes	Medium	Medium	Low (no critic)

- ▶ A **critic** V_ϕ learns to estimate state values via TD; the TD residual δ_t approximates A^π cheaply
- ▶ **A2C** parallelises data collection with N synchronous actors; A3C uses asynchronous workers (empirically $A2C \geq A3C$)
- ▶ **TRPO** guarantees monotonic improvement via a KL trust region but is expensive to compute
- ▶ **PPO-Clip** approximates the trust region with a first-order method: clip $r_t(\theta)$ to $[1-\varepsilon, 1+\varepsilon]$ and take the min with the unclipped objective
- ▶ PPO is the workhorse of modern on-policy RL: simple, stable, widely used in robotics, games, and RLHF
- ▶ **GRPO** drops the critic and uses group-mean baselines — now the default for LLM RL (DeepSeek-R1)

- ▶ **Function approximation bias:** the TD residual δ_t is only an unbiased advantage estimate if $V_\phi = V^\pi$. An inaccurate critic introduces bias into the policy gradient
- ▶ **Two-network instability:** if the critic lags behind the actor (or vice versa), the feedback loop can diverge — both networks must be trained carefully together (GRPO sidesteps this by dropping the critic entirely)
- ▶ **Hyperparameter sensitivity:** PPO performance is sensitive to ε (clip range), λ (GAE), number of epochs per rollout, and entropy coefficient c_2
- ▶ **On-policy sample inefficiency:** all methods in this lecture discard experience after each update. In complex environments this can require billions of environment steps
- ▶ **Dense-reward assumption:** variance reduction via advantage estimation works well when rewards are frequent. Sparse-reward tasks remain challenging for all on-policy methods

- ▶ **Key limitation:** on-policy methods discard all experience after each update
— data-hungry for complex environments

- ▶ **Key limitation:** on-policy methods discard all experience after each update — data-hungry for complex environments
- ▶ **Day 5 — DDPG & TD3:** off-policy deterministic policies for continuous control — replay buffer, action-value critic $Q_\phi(s, a)$, gradient through $Q_\phi(s, \mu_\theta(s))$
- ▶ **Day 6 — SAC:** max-entropy Actor-Critic — the same $c_2 \mathcal{H}(\pi_\theta)$ bonus you saw in PPO, now **central** to the objective
- ▶ **Day 7 — Exploration:** that entropy bonus *is* exploration via a stochastic policy; Day 7 makes exploration the explicit objective
- ▶ **Day 9 — RLHF:** PPO is the algorithm underneath the original InstructGPT/ChatGPT pipeline; GRPO is its critic-free successor

Policy Optimization: **References**

- [1] Sutton, R. S., & Barto, A. G. (2018).
Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

- [2] Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (2000).
Policy Gradient Methods for Reinforcement Learning with Function Approximation.

In *Advances in Neural Information Processing Systems (NeurIPS)* 12, 1057–1063.

- [3] Kakade, S., & Langford, J. (2002).
Approximately Optimal Approximate Reinforcement Learning.

In *Proceedings of the 19th International Conference on Machine Learning (ICML)*.

- [4] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016).

Asynchronous Methods for Deep Reinforcement Learning.

In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, PMLR 48, 1928–1937.

- [5] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015).

Trust Region Policy Optimization.

In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.

- [6] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016).
High-Dimensional Continuous Control Using Generalized Advantage Estimation.
In International Conference on Learning Representations (ICLR).
- [7] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017).
Proximal Policy Optimization Algorithms.
arXiv preprint arXiv:1707.06347.
- [8] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).
Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.
In Proceedings of the 35th International Conference on Machine Learning (ICML), PMLR 80, 1856–1865.

- [9] Andrychowicz, M., Raichuk, A., Stańczyk, P., et al. (2020).
What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study.
arXiv preprint arXiv:2006.05990.

- [10] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., et al. (2024).
DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.
arXiv preprint arXiv:2402.03300. (Introduces GRPO.)

- [11] DeepSeek-AI. (2025).
DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.
arXiv preprint arXiv:2501.12948.

[12] OpenAI Baselines.

<https://github.com/openai/baselines>

[13] Spinning Up by OpenAI.

<https://spinningup.openai.com>

[14] Sergey Levine, Berkeley CS285: Deep Reinforcement Learning.

<https://rail.eecs.berkeley.edu/deeprlcourse/>

[15] Chelsea Finn & Karol Hausman, Stanford CS224R: Deep Reinforcement Learning.

<http://cs224r.stanford.edu/>

[16] Emma Brunskill, Stanford CS234: Reinforcement Learning.

<https://web.stanford.edu/class/cs234/>

Credits

Dr. Prashant Aparajeya

Computer Vision Scientist — Director(AISimply Ltd)

p.aparajeya@aisimply.uk

This project benefited from external collaboration, and we acknowledge their contribution with gratitude.

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

- Reference only — students implement PPO, not TRPO, in the lab (see Schulman et al., 2015)